# Register Machines

Guillermo Polito

















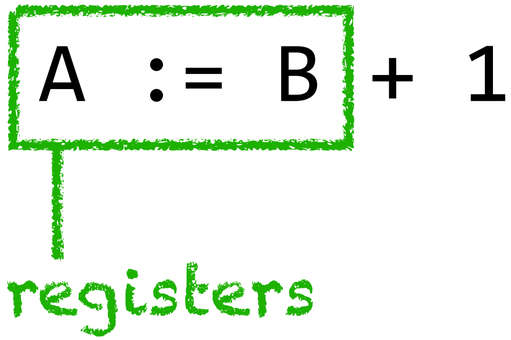
# Goals

- Understand register machines
- Two-address code vs Three-address code
- Machine code and assembly code
- Instruction sets, the RISC and CISC models
- Comparisons and control flow
- Instruction encoding

# Register Machines

- Operands are exchanged through *explicit registers*

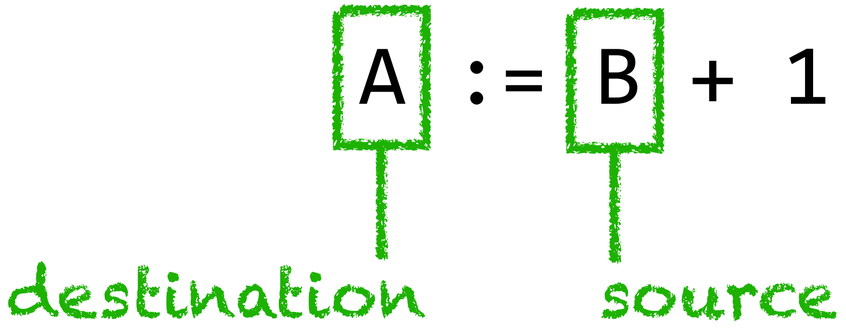
- Each machine defines its own set of registers
  - Could be fixed or *infinite*
  - Could be physical or virtual/symbolic



# Register Instruction Sets

- Many different machines exist (x86, ARM, abstract machines…)
- Each define an programming model named *Instruction Set Architecture*

- Different instruction set *designs*
  - Operand design: 3AC and 2AC
  - Set complexity: RISC vs CISC
  - Encoding: fixed size vs variable size



# Two-Address-Code

## 2AC, two-address-instruction

- One register is both source and destination
- Warning: one operand register gets overwritten!
- Destination could be at the left (aka intel notation)

```
ADD A, B
```
equivalent to:
```
A := A + B
```

- Destination could be at the right (aka AT&T notation)

```
ADD B, A
```
equivalent to:
```
A := B + A
```

# Two-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
LOAD t1 a

LOAD t2 b

ADD t1 t2

LOAD t2 3

DIV t2 t1

CEILING t2
```

| t1 | t2 |
| --- | --- |
| | |

# Two-address Code Example

```
(3 / (a + b)) . ceiling()
```



```
LOAD t1 a

LOAD t2 b

ADD t1 t2

LOAD t2 3

DIV t2 t1

CEILING t2
```

| t1 | t2 |
|---|---|
| 17 | |

# Two-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
LOAD t1 a
LOAD t2 b
ADD t1 t2
LOAD t2 3
DIV t2 t1
CEILING t2
```

| t1 | t2 |
|---|---|
| 17 | 42 |

# Two-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
LOAD t1 a
LOAD t2 b
ADD t1 t2
LOAD t2 3
DIV t2 t1
CEILING t2
```

| t1 | t2 |
|---|---|
| 59 | 42 |

# Two-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
LOAD t1 a

LOAD t2 b

ADD t1 t2

LOAD t2 3

DIV t2 t1

CEILING t2
```

| t1 | t2 |
|---|---|
| 59 | 3 |

# Two-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
LOAD t1 a
LOAD t2 b
ADD t1 t2
LOAD t2 3
DIV t2 t1
CEILING t2
```

| t1 | t2 |
|---|---|
| 59 | 0.05… |

# Two-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
LOAD t1 a

LOAD t2 b

ADD t1 t2

LOAD t2 3

DIV t2 t1

CEILING t2
```

| t1 | t2 |
|---|---|
| 59 | 1 |

# Three-Address-Code

## TAC, 3AC

- Instruction have generally three operands: one destination and two sources
- Many notations you'll find in the wild:

```
A := B + C
```

- Destination could be at the left (aka intel notation)

```
ADD A, B, C
```

- Destination could be at the right (aka AT&T notation)

```
ADD B, C, A
```

# Three-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
t1 := LOAD a

t2 := LOAD b

t1 := ADD t1 t2

t2 := 3

t2 := t2 / t1

t2 := CEILING t2
```

| t1 | t2 |
| --- | --- |
| | |

# Three-address Code Example

```
(3 / (a + b)) . ceiling()
```



```
t1 := LOAD a
t2 := LOAD b
t1 := ADD t1 t2
t2 := 3
t2 := t2 / t1
t2 := CEILING t2
```

| t1 | t2 |
|---|---|
| 17 | |

# Three-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
t1 := LOAD a
t2 := LOAD b
t1 := ADD t1 t2
t2 := 3
t2 := t2 / t1
t2 := CEILING t2
```

| t1 | t2 |
|---|---|
| 17 | 42 |

# Three-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
t1 := LOAD a
t2 := LOAD b
t1 := ADD t1 t2
t2 := 3
t2 := t2 / t1
t2 := CEILING t2
```

| t1 | t2 |
|---|---|
| 59 | 42 |

# Three-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
t1 := LOAD a
t2 := LOAD b
t1 := ADD t1 t2
t2 := 3
t2 := t2 / t1
t2 := CEILING t2
```

| t1 | t2 |
|---|---|
| 59 | 3 |

# Three-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
t1 := LOAD a
t2 := LOAD b
t1 := ADD t1 t2
t2 := 3
t2 := t2 / t1
t2 := CEILING t2
```

| t1 | t2 |
|---|---|
| 59 | 0.05… |

# Three-address Code Example

```
(3 / (a + b)) . ceiling()
```

```
t1 := LOAD a
t2 := LOAD b
t1 := ADD t1 t2
t2 := 3
t2 := t2 / t1
t2 := CEILING t2
```

| t1 | t2 |
|---|---|
| 59 | 1 |

# The CISC and RISC Models

- **CISC:** Complex Instruction Set Computer
  - Denotes a machine with lots of instructions, complex encoding
  - Examples: x86, AMD
- **RISC:** Reduced Instruction Set Computer
  - Denotes a machine with few instructions, simple encoding
  - Examples: ARM, RISC-V

# Conditional Jumps and Status Registers

- Register machines store the result of comparisons in a **Flag/Status register**
- Status registers typically contain *binary flags* with comparison information
  - Some instructions *set/unset* some flags
  - Other *read* the flags
  - **Most do not modify nor read** them

**Status Register**

| | |
|---|---|
| > | 0 |
| < | 1 |
| = | 0 |
| overflow | 1 |
| … | … |

# Status Registers in different ISAs

- In some architectures (e.g., x64, ARM64) the status register is *a special register, implicitly used*

```
CMP A, B

JumpIfGreaterThan targetLabel
```

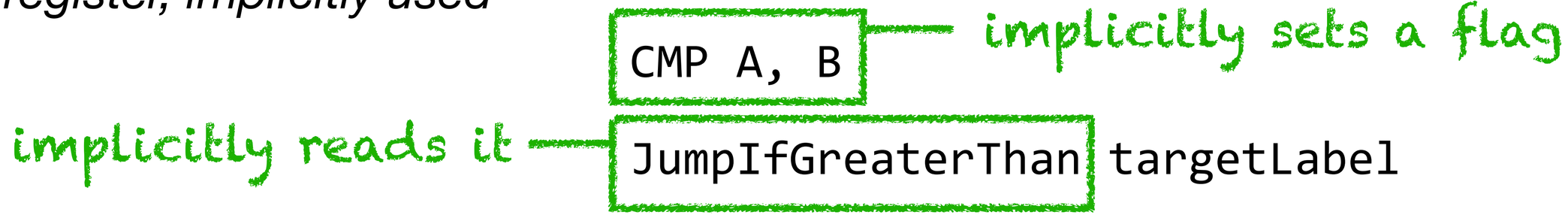
- In others (RISC-V, MIPS), it is *explicit, a general purpose register*

```
RESULT := CMP A, B

JumpIfGreaterThan RESULT targetLabel
```

# Status Registers in different ISAs

- In some architectures (e.g., x64, ARM64) the status register is *a special register, implicitly used*




- In others (RISC-V, MIPS), it is *explicit, a general purpose register*



# Machine Code and Physical Machines

- Binary code executed by a machine: a CPU, GPU…
- Think of CPUs as *very fast interpreters* of machine code

- Each physical machine has its own ISA and binary encoding of it
- Typically register machines, although support some stack operations

```
#[ 31 32 3 213 76 1 0 88 ]
```

some arm v8 instructions

# Machine Code VS Assembly Code

- Assembly is a programming language
- (not machine code)
- Translated to machine code using a compiler (aka an assembler)

```
nop
ldr x12, #40
```

```
#[ 31 32 3 213 76 1 0 88 ]
```

```
nop          ldr x12, #40
```

**For simplicity: we will use assembly examples to represent machine code**

# Machine Code Instructions

- Basic instructions:
  - Write data to memory
  - Read data from memory
  - Arithmetic ( + , - , * …)
  - Bit instructions (shift, bitAnd …)
  - Control flow (jump, jump if)

```
mnemonic op1 op2 op3 …
```

```
e.g.,
  load r1 [r1, #40]
  add r3 r2 r1
  store [r1, #40] r3
```

# Machine Code Instruction Operands

## Basic operand types

- Immediate numbers: encoded directly in the instruction bytes

  e.g., `#40`

- Registers: small memories in the CPU

  e.g., `r1 r2 r3`

- Memory addresses: calculated from registers and/or immediates

  e.g., `[r1, #40]`

```
load r2 [r1, #40]
add r3 r2 r1
store [r1, #40] r3
```

# Machine Code Registers

- Small memories in the CPU
- Some are for general use
- Some are specific (e.g., Floating point numbers)
- Some are for the CPU (and not for us)

=> And there are very few!!!

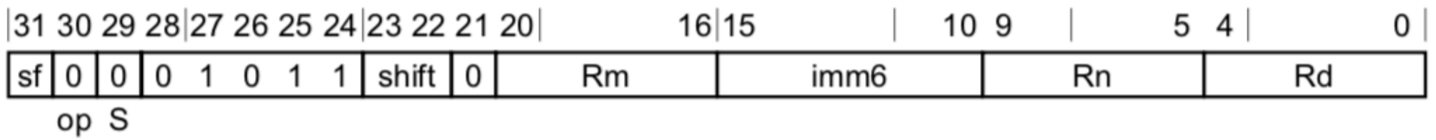
```
e.g.,
  load r1 [r1, #40]
  add r3 r2 r1
  store [r1, #40] r3
```

# Machine Code Binary Encoding

## Each ISA has its manual

### C6.2.5 ADD (shifted register)

Add (shifted register) adds a register value and an optionally-shifted register value, and writes the result to the destination register.




***32-bit variant***

Applies when `sf == 0`.

```
ADD <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

***64-bit variant***

Applies when `sf == 1`.

```
ADD <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Arm® Architecture Reference Manual

**Armv8, for Armv8-A architecture profile**

# To go deeper

- Some CPUs have special registers that are used with specific instructions
  - SIMD, floating point (FP)…
  - The registers we saw today are called *general purpose (GP)*
  - Moving data from GP to FP, SIMD requires specific instructions
- Be careful! Not only comparisons modify the status register
  - Check Additions, Subtractions…

# Takeaways

- In Register based machines operands are explicit

- Registers represent variables
- Different machines present different designs
  - e.g., infinite or fixed, physical or symbolic registers
- Three-address-code has explicit destination
- Two-address-code has implicit destination